CDS528 GROUP PROJECT

SafeVendor

A trust-minimized token sale with direct 2-of-3 treasury settlement

Repository: https://github.com/aurora371 624-star/safevendor

1,000,000	2 of 3	0.0001 ETH	17
fixed SVT supply	treasury approvals	price per SVT	automated tests

Item	Submission information
Course	CDS528 Blockchain DeFi Smart Contract Development
Group	SafeVendor Group
Members	Yang Qingchen, Xu Fang, Wang Zijian, Wu Fei, Chen Qikai
Network	Ethereum Sepolia, chain ID 11 155 111
Submission	18 November 2026

SafeVendor addresses a concrete token-sale custody problem: buyers must currently trust a seller or operator to hold sale proceeds and later honor withdrawals. The design removes that trust window by sending ETH directly to a fixed-owner multisig treasury in the same transaction that delivers the token. The deployed system is test-only, publicly verified, and fully documented for reproduction.

1. Problem Statement

1.1 Context and users

The primary users are small project teams or DAOs that want a simple fixed-price token sale, and buyers who want assurance that sale proceeds do not remain under one person's control. The secondary user is the three-person treasury group responsible for spending proceeds after the sale.

A conventional custody flow asks buyers to send ETH to a seller-controlled address, after which the seller promises to transfer the funds into a shared account. The buyer cannot enforce that later step. A database can record the promise, but it cannot make the subsequent transfer non-discretionary.

1.2 Problem statement

How can a fixed-price ERC-20 sale guarantee that every successful purchase settles into shared custody without allowing a seller, deployer, or single treasury member to redirect or withdraw the proceeds unilaterally?

1.3 Why a smart contract

- Settlement is atomic: the payment, treasury deposit, and token delivery either all succeed or all revert.
- The 2-of-3 threshold is enforced by code rather than by an off-chain policy.
- Token supply, price, owners, and threshold are publicly inspectable and cannot be silently changed after deployment.
- Buyers can verify the deployed bytecode and contract state on Etherscan.

1.4 Goals and scope

Goal	Success condition
Direct settlement	Every successful buy sends ETH to Treasury with no seller withdrawal path.
Shared control	Two distinct owners must confirm before any Treasury transfer executes.
Safe failure	Wrong payment, insufficient inventory, and failed transfers revert.
Reproducibility	A new user can install, test, deploy, and verify from the README.

1.5 Constraints and assumptions

- Sepolia only; all assets are test assets with no monetary value.
- Fixed supply of one million SVT; there is no post-deployment mint.
- Three fixed treasury owners and a threshold of two.
- No proxy, upgrade path, pause switch, or owner replacement.
- The sale price does not follow a market oracle; it is a teaching mechanism.

2. Solution Design

2.1 Architecture and roles

SafeVendor separates three responsibilities. The token contract owns only the fixed supply. The vendor owns the sale rule and inventory. The treasury owns shared custody and transaction approval. No contract can upgrade or replace another contract after deployment.

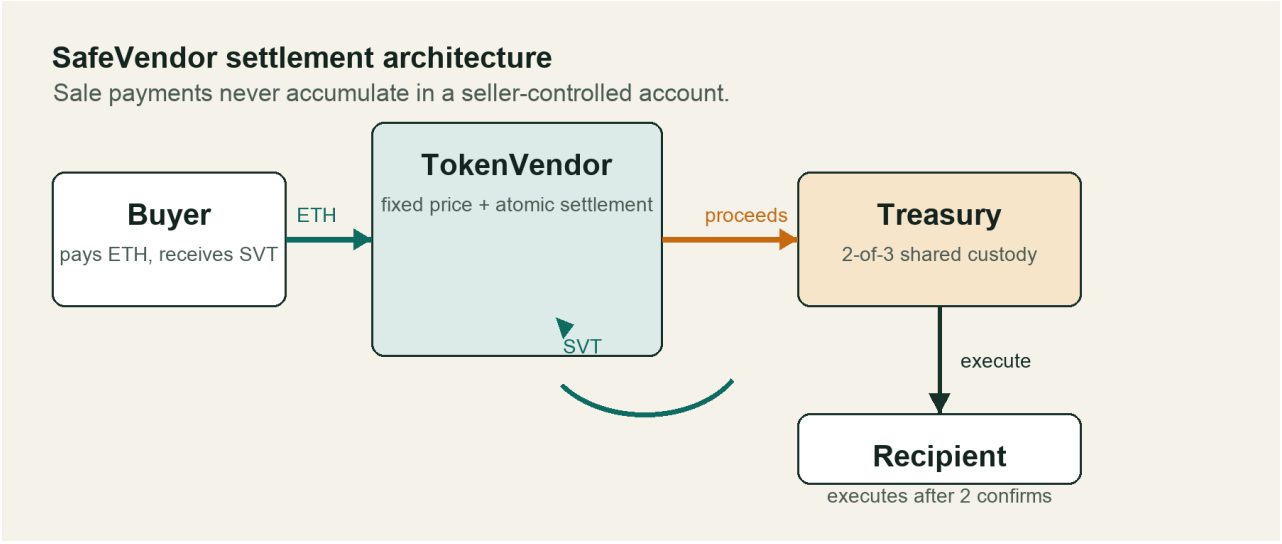


Figure 1. Sale payments settle directly in the multisig treasury.

Actor	On-chain capability	Trust boundary
Buyer	Calls buyTokens with ETH and a minimum SVT output.	Does not need to trust a seller address with custody.
Treasury owner	Submits, confirms, and executes proposals when threshold is met.	One owner is insufficient; two can collude.
Seller	Supplies token inventory; has no vendor withdrawal function.	Cannot retrieve sale ETH from the vendor.
Frontend	Reads state and prepares wallet transactions.	Cannot bypass contract access control.

2.2 Mechanism and token rationale

ERC-20 is appropriate because the sale needs a fungible unit that any wallet can hold and transfer. SafeVendorToken uses OpenZeppelin ERC20 and mints the entire one-million-token supply once. It deliberately omits owner-only mint and burn functions, preventing post-sale supply dilution.

The vendor stores the token address, treasury address, and price as immutable values. A fixed price is feasible for this teaching scope and avoids oracle complexity. The design's distinctive property is not a new token standard. It is the settlement combination: a familiar vendor pattern whose proceeds flow into a multisig that no single operator controls.

2.3 Key process flows

Purchase flow	Treasury flow
1. Buyer sends ETH and minTokensOut.	1. Owner submits a recipient, value, and calldata.

Purchase flow	Treasury flow
2. Vendor quotes the fixed-price token amount.	2. Submission records the first owner confirmation.
3. Vendor checks inventory and slippage.	3. A different owner confirms the proposal.
4. Vendor sends net ETH to Treasury.	4. Execution requires confirmation count >= 2.
5. Vendor transfers SVT and emits an event.	5. Executed is set before the external call.

2.4 On-chain and off-chain partition

Trust-critical state and value transfer stay on-chain: balances, ownership, confirmations, execution state, and ETH movement. The optional Vite frontend stays off-chain because it only reads chain state and prepares signed transactions. There is no server-side custodian or backend signing key.

3. Implementation

3.1 Core mechanisms

Mechanism	Implementation	Why it matters
Fixed supply	One constructor mint; immutable maxSupply.	No post-deployment dilution.
Fixed price	Math.mulDiv converts ETH amount to token units; minimum output protects the buyer.	Deterministic sale terms.
Direct settlement	Checked native call forwards net ETH to Treasury before token delivery.	No seller custody window.
2-of-3 execution	Per-owner confirmation mapping and immutable threshold.	One owner cannot move funds.
Replay protection	Executed state is set before the external call.	A transaction cannot execute twice.
Reentrancy protection	CEI plus OpenZeppelin ReentrancyGuard.	External calls cannot re-enter value paths.

3.2 Events and access control

Token and vendor behavior uses standard ERC-20 Transfer events and a custom TokensPurchased event with gross payment, net settlement, refund, and token amount. Treasury emits Deposit, TransactionSubmitted, TransactionConfirmed, and TransactionExecuted. The event set is sufficient to reconstruct each purchase and approval sequence from logs.

Treasury submission, confirmation, and execution are guarded by onlyOwner. The owner mapping and fixed owner array are initialized once. The vendor has no privileged withdrawal function, so even the deployer cannot retrieve its sale ETH.

3.3 Gas-awareness and code quality

- Immutable token, treasury, price, threshold, and max supply avoid storage reads.
- The only loop is bounded owner initialization at deployment time.
- Small custom errors replace long revert strings.
- The optimizer is enabled with 200 runs.
- Events expose value-bearing state changes without requiring historical storage.

- Code is split by responsibility and uses OpenZeppelin components rather than reimplementing token, transfer, and reentrancy primitives.

3.4 Frontend integration

The dApp displays live Sepolia state, quotes a purchase, collects wallet signatures, submits treasury proposals, and exposes owner confirm and execute actions. It reads deployment metadata from a generated JSON file, so the interface cannot silently point at a different contract set.

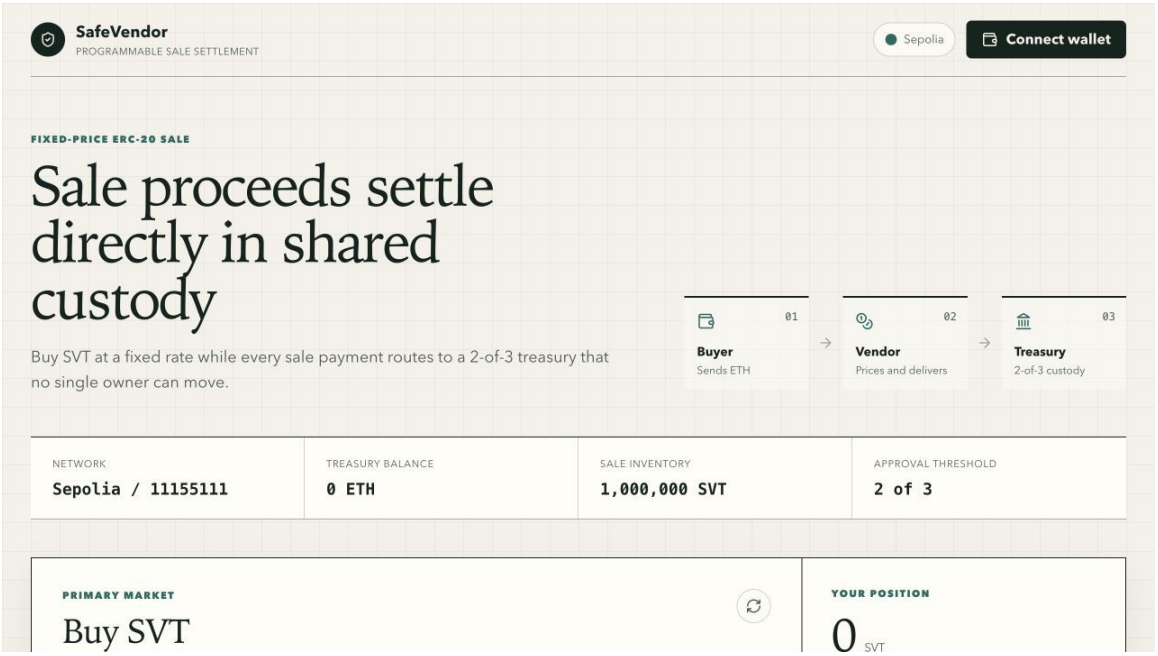


Figure 2. Operational dApp displaying live Sepolia sale and treasury state.

4. Testing and Security

4.1 Verification results

17	100%	100%	2
tests passing	production line coverage	function coverage	Slither INFO findings

The test suite exercises successful behavior and failure paths, including duplicate confirmation, unauthorized callers, replay, slippage, insufficient inventory, invalid configuration, rounding refunds, and reentrancy. Production contracts have 100 percent statement, function, and line coverage. Aggregate branch coverage is 79.03 percent.

4.2 Static analysis

Slither analyzed 18 compiled contracts with 100 detectors after excluding OpenZeppelin dependencies and the test-only receiver. It reported no High or Medium severity findings. The two project-level findings are informational low-level call findings.

Finding	Location	Decision
Low-level call	MultiSigTreasury.executeTransaction	Required for arbitrary approved calldata and contract wallets; success is checked.
Low-level call	TokenVendor.buyTokens	Required for ETH settlement and refunds; both return flags are checked.

4.3 Top threats and mitigations

Threat	Attack path	Mitigation and residual risk
Single-owner exfiltration	One compromised owner submits and executes a transfer.	2-of-3 threshold blocks one owner; two colluding owners remain a residual risk.
Reentrancy	Recipient re-enters during ETH delivery.	CEI plus ReentrancyGuard; the reentry test confirms rejection.
Replay	Execute the same approved transfer twice.	Executed is checked and set before the call; replay reverts.
Duplicate approval	One owner confirms repeatedly to reach threshold.	A per-owner, per-transaction confirmation mapping blocks duplicates.
Unauthorized action	Non-owner submits, confirms, or executes.	OnlyOwner on all treasury state changes.
Payment ambiguity	Buyer overpays or price rounding differs.	Minimum output and explicit remainder refund; failed sends revert.

4.4 Economic and compliance risks

- The fixed price is not market-derived and can diverge from any external value.
- Three EOAs create key-management risk; the design assumes independent custody.
- The absence of an upgrade path prevents patching; migration requires redeployment.
- There is no owner-removal mechanism; lost keys reduce availability.
- The project must remain on Sepolia and must not be presented as an investment.

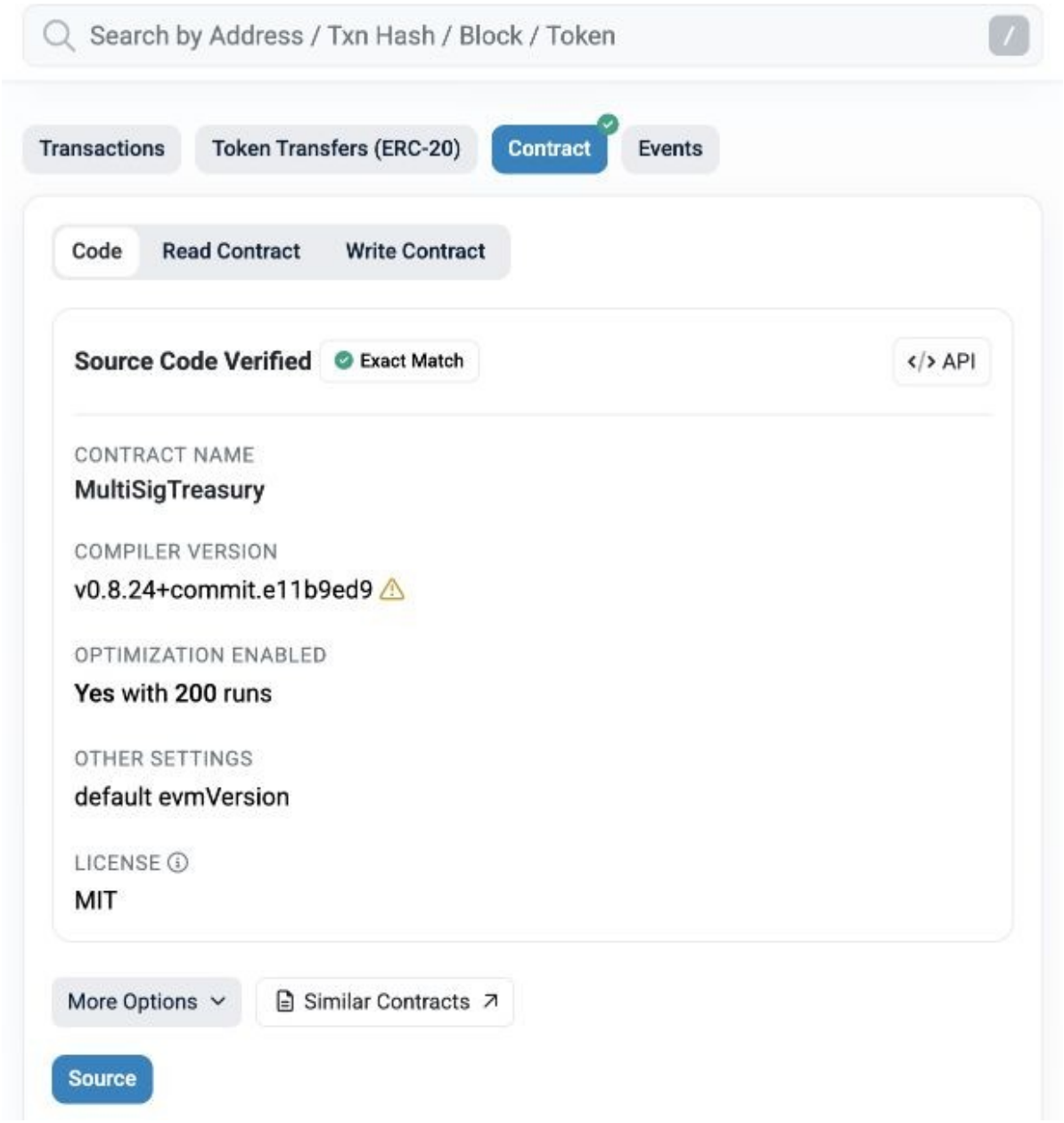


Figure 3. Etherscan reports matching bytecode and ABI for the treasury.

5. Deployment and DevOps

5.1 Live Sepolia deployment

The contracts are deployed on Sepolia and verified on Etherscan with Solidity 0.8.24, optimizer enabled, and 200 runs. Constructor arguments match the live deployment and the source is available through Standard JSON verification.

Contract	Verified Sepolia address
SafeVendorToken	0x734c0e5a46 624Ef79b9bcb6283f9537F3018B79 B
MultiSigTreasury	0xaCCb68DE59C32 300BB224 525E05B90b9f69 6 17B4
TokenVendor	0x96fAbD18cfEC365 700b7D95c397b4AdF04 274 717

5.2 Reproducible deployment

pnpm deploy validates chain ID 11 155 111, deploys Token, Treasury, and Vendor in dependency order, transfers the full inventory, and writes addresses, transaction hashes, and constructor data to deployments/sepolia.json. The frontend imports the generated deployment file so UI state and on-chain contracts stay aligned.

Artifact	Purpose
pnpm compile	Compiles Solidity 0.8.24 with optimized settings.
pnpm test	Runs 17 unit and failure-path tests.
pnpm coverage	Writes Istanbul coverage reports.
pnpm deploy	Deploys and records addresses and receipts.
pnpm verify	Verifies constructors through the Etherscan API.
pnpm frontend:build	Creates the production Vite bundle.

5.3 Operational readiness

The contracts deliberately omit pause and upgrade controls. This choice removes a privileged emergency key and prevents an administrator from freezing the sale or replacing the treasury. The trade-off is that a logic defect cannot be patched in place; the team would deploy a new immutable contract set and inform users.

Operationally, two of three owner keys must remain available. Only one may be offline without blocking execution. Two offline or lost keys halt treasury spending, while two compromised keys can collude. These conditions are stated explicitly rather than hidden behind an admin recovery function.

5.4 Live deployment evidence

Event	Observed result
Owner 1	0x758 090f1c15a76fFf31C5852e71f68ABCF24FA61
Owner 2	0xAD8C645 384 858B50b988BFEbb0c1DB6f996C5A76

Event	Observed result
Owner 3	0x50b74 325eadd91 918d9f4 083 635d9bB570dC9C56
Threshold	2 of 3 confirmations required.
Vendor inventory	1,000,000 SVT received at deployment.
Source verification	All three contracts match Etherscan Standard JSON input.

6. Documentation, Process, and Evaluation

6.1 Repository and documentation

The repository includes source, tests, deployment scripts, a reproducible README, security evidence, a requirements matrix, a demo script, and the optional frontend. The readme identifies exact prerequisites, network, commands, risk controls, and the operational rationale for immutability.

The public GitHub URL is on the cover and at the top of the repository README. The cover lists the five group members. The final group ID must be confirmed before Moodle submission.

6.2 Rubric coverage

Criterion	Evidence	Risk
R1 Problem Statement	Concrete users, custody pain point, why on-chain, scope and assumptions.	Low
R2 Solution Design	Three-contract architecture, actor table, rationale, purchase and approval flows.	Low
R3 Implementation	Deployed contracts, events, access control, immutable values, frontend.	Low
R4 Testing and Security	17 tests, full production line coverage, Slither, threat table.	Low
R5 Deployment and DevOps	Three verified Sepolia addresses, reproducible scripts, immutability rationale.	Low
R6 Documentation	README, report, citations, license, AI disclosure.	Medium until group metadata and presenter names are inserted.
R7 Presentation	Editable slide deck, 12-minute flow, four speaker roles, Q&A preparation.	Medium until real member roles are assigned and rehearsed.

6.3 Team process

- Project lead: scope, rubric mapping, submission, and timeline control.
- Smart contract lead: contract design, deployment, verification, and gas review.
- Testing and security analyst: failure-path tests, Slither, assumptions, and Q&A.
- Frontend and integration engineer: Vite + ethers.js workflow in the dApp, wallet flows, and live demonstration.
- All members participate in final rehearsal and answer at least one question.

6.4 Limitations and future work

The current project intentionally favors a small auditable scope. Future work could add an explicit inventory recovery decision, owner rotation with timelocks, proposal cancellation, richer event

indexing, fuzz invariants, and a formal threat model for market pricing. Those additions would introduce new trust surfaces and should not be added without matching tests and governance rationale.

7. Conclusion

SafeVendor meets the assignment's central engineering goal. It presents a concrete DeFi custody problem, implements a complete Solidity system, tests failure paths, deploys and verifies on Sepolia, provides a usable frontend, and documents the operational trade-offs. The architecture is deliberately small: three contracts with clear ownership boundaries and no hidden administrator.

The strongest evidence is the live Sepolia state: the group owner is fixed at deployment, the treasury requires a second independent owner, the vendor holds the complete token inventory, and Etherscan shows matching source and ABI for all three contracts.

The public repository is attached on the cover. The remaining administrative step is to insert the final formal group ID before Moodle submission.

References

Ethereum Foundation. (2026). Ethereum developer tutorials. https://ethereum.org/developers/tutorials/

Ethereum Foundation. (2026). ERC-20 token standard. https://ethereum.org/developers/docs/standards/tokens/erc-20/

Etherscan. (2026). Ethereum Sepolia block explorer. https://sepolia.etherscan.io/

Nomic Foundation. (2026). Hardhat documentation. https://hardhat.org/docs

OpenZeppelin. (2026). OpenZeppelin Contracts 5.x. https://docs.openzeppelin.com/contracts/5.x/

Solidity Team. (2024). Solidity 0.8.24 documentation. https://docs.soliditylang.org/en/v0.8.24/

Trail of Bits. (2026). Slither static analysis. https://github.com/crytic/slither

Appendix A. Deployment Addresses

Contract	Verified address	Deployment transaction
SafeVendorToken	0x734c0e5a46 624Ef79b9bcb628 3f9537F3018B79B	0xe2ee577 305b2bc4 328 241fab13fde9d69 255d0bf0 ae0c9ca64 238c6 544 024fb0
MultiSigTreasury	0xaCCb68DE59C32 300BB224 5 25E05B90b9f69 617B4	0x7a3 099 374ecd0aaea895cd12ba6150fa52bc0369d d0e73d9db14c87a2c4f16c4
TokenVendor	0x96fAbD18cfEC365 700b7D95 c397b4AdF04 274 717	0xa82 989eeedf57f3637bfdf4e012da63 176 561d8da4 bec6883a9e4d8d52 828f32